A R F L

Privacy in the Dark Cloud

A Decentralised Privacy Protocol Powered by Bitcoin

---

arfl.io · github.com/arfl-protocol · #arfl on Nostr

Abstract. *The VPN industry sold privacy. What it actually sold was trust in a company — a company with servers that can be seized, a CEO that can be pressured, and invoices that link identity to browsing history. ARFL is a decentralised privacy protocol that eliminates all three problems. ARFL aggregates four open protocols — WireGuard for encrypted transport, Nostr for pseudonymous node discovery, the Bitcoin Lightning Network for payment, and Fedimint for trustless deposit custody and peer-to-peer settlement — into a self-sustaining privacy network that anyone can build on. Node operators plug directly into the ARFL protocol, earning Bitcoin passively from bandwidth they already own. Hubs are products built on top of the protocol: they set their own prices and compete on brand and service — the protocol itself never takes a cut, in any version, under any circumstance. A user's WireGuard keypair — generated locally and never transmitted — is their sole identity. Nobody owns the network. Nobody controls the network. The protocol defines the rules. Mathematics enforces them.*

Draft v0.2 · 2026 · No rights reserved

1. Introduction

Commerce on the internet has come to rely almost entirely on centralised intermediaries to provide privacy services. The VPN industry, despite its promises, reproduces the same structural problem: a central company holds the keys, the logs, and the liability. That company can be subpoenaed, hacked, acquired, or pressured. Its users have no recourse and no verification. They are asked to trust. Trust is not privacy.

The subscription model compounds the problem. Monthly payments require credit cards. Credit cards require identity. Users sign up at promotional rates and renew at multiples of the original price — surprised, locked in, and no more private than before. Every centralised VPN, regardless of its privacy claims, can be subpoenaed, hacked, acquired, or pressured into policy changes. The architecture is the vulnerability.

What is needed is a privacy network based on cryptographic proof and economic incentive rather than trust — one where no central server exists to seize, no account exists to subpoena, and no company exists to pressure. In this paper we propose ARFL: a decentralised privacy protocol that aggregates four existing open protocols into a self-sustaining privacy network owned by nobody and available to everyone.

ARFL defines two classes of participant. Node operators interact directly with the protocol, contributing bandwidth and earning Bitcoin. Hubs are products built on top of the protocol, providing user-facing experiences and competing on brand and service quality. The protocol itself owns no infrastructure, controls no funds, and requires no permission to build on. Between users and node operators sit no intermediary the protocol defines — a hub is simply a business built on the protocol, in the same sense a company is a business built on AWS or on Stripe. The protocol has no visibility into, and no opinion on, how a hub makes money.

The key properties of ARFL are: (1) no user accounts or identity — a locally generated WireGuard keypair is the user's sole credential; (2) no central server — node operators publish availability pseudonymously via Nostr; (3) no subscription — users purchase bandwidth bundles via Bitcoin Lightning at a price the hub sets freely; (4) no logs — the architecture makes logging impossible, provable by inspection of open source code; (5) no token — the incentive mechanism is Bitcoin; (6) no protocol fee — every satoshi a user pays passes through to the node operators who route their session, in v1 and in every future version.

2. System Architecture

ARFL is composed of four protocol layers and two participant layers. No new cryptography is introduced. The contribution of ARFL is the composition of existing protocols into a coherent privacy system with aligned economic incentives, and a settlement design in which the protocol itself is structurally incapable of collecting a fee.

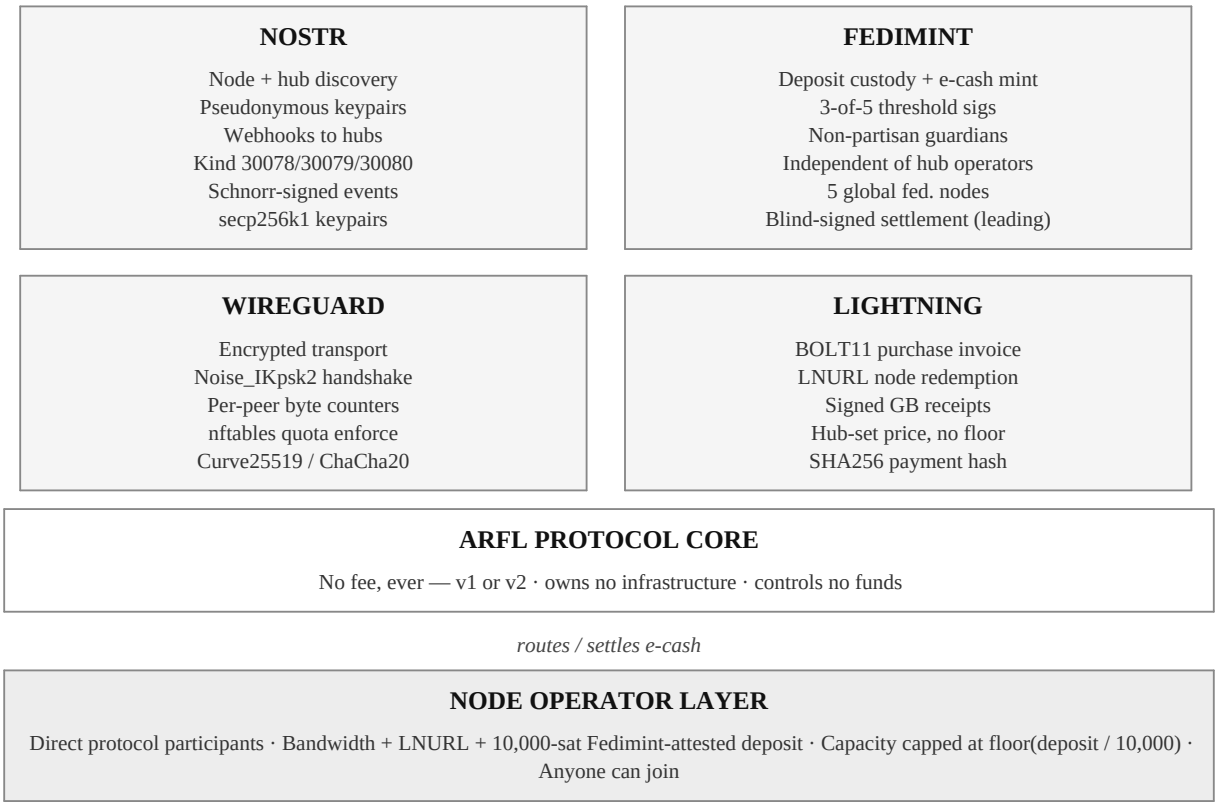
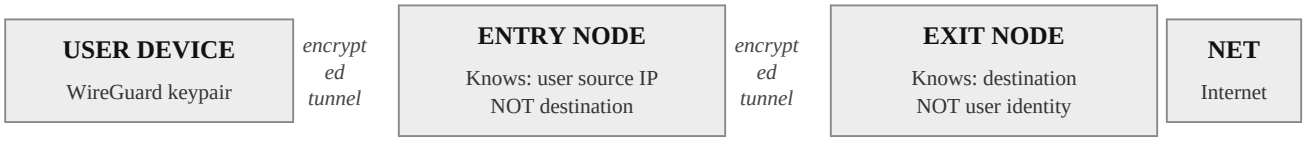


Figure 1. ARFL protocol architecture. The protocol core owns nothing and collects no fee; the hub sets price and the routing nodes are paid in full.

The session data flow proceeds through a mandatory two-hop routing design in v1. No single node holds the complete picture of a user session:



The ARFL client selects and pairs entry and exit nodes. Neither node is informed of the other's identity. All routing decisions are made unilaterally by the client. DNS queries route through the tunnel to Quad9 (9.9.9.9) — no DNS leak possible.

Figure 2. Two-hop session routing.

3. Cryptographic Primitives

ARFL inherits its cryptographic security from four independent protocol stacks, each well-audited and widely deployed in production systems.

3.1 WireGuard

WireGuard implements the Noise_IKpsk2 handshake protocol, providing mutual authentication and forward secrecy. The cryptographic primitives are fixed and non-negotiable, eliminating downgrade attacks by design:

- **Curve25519** — Elliptic-curve Diffie-Hellman key exchange.
- **ChaCha20-Poly1305** — Authenticated encryption (AEAD) for all tunnel traffic.
- **BLAKE2s** — Cryptographic hashing for key derivation and MAC operations.
- **SipHash24** — Hashtable keying to prevent denial-of-service via hash collision.

Each WireGuard peer is identified by a 256-bit Curve25519 public key. The ARFL client generates a keypair locally on first launch and stores the private key in the device secure enclave. This keypair is the user's sole persistent identity — no username, no email, no password.

3.2 Nostr

Nostr uses secp256k1 elliptic curve keypairs for identity. Every node and hub announcement is a Nostr event signed with a Schnorr signature (BIP-340) over the SHA256 hash of the canonical event serialisation (NIP-01). Operators are identified solely by their Nostr public key — no real-world identity is required or stored.

```
Event ID: SHA256(canonical_json_serialisation)
Signature: Schnorr(private_key, event_id) -- BIP-340, Bitcoin-Taproot-compatible
Encoding: NIP-19 bech32 (npub / nsec) for display
Transport: WebSocket to Nostr relays
```

3.3 Lightning Network

Bundle purchases are made via BOLT11 invoices. Each invoice encodes a SHA256 payment hash derived from a secret preimage known only to the payee. Payment is cryptographically proven by revealing the preimage. The entire ARFL economy is denominated in satoshis. USD figures anywhere in this document are human-readable references only and carry no protocol obligation.

3.4 Fedimint

Fedimint provides two functions in ARFL: federated custody of node operator security deposits, and — under the settlement design described in Section 6 — blind-signed e-cash used to pay routing node operators without the hub ever holding their funds. The ARFL protocol team operates a Fedimint federation of five nodes across separate global jurisdictions, requiring three-of-five signatures to release or slash any deposit. This eliminates single-operator control of node operator funds while remaining completely transparent to participants — the deposit address is indistinguishable from any Lightning address.

Governance requirement: Fedimint guardians must be non-partisan and operationally independent of any hub operator, including the ARFL protocol team itself if it runs its own reference hub. Without this separation, a hub that also held guardian keys could in principle link its own minted e-cash notes to their later redemption — breaking the unlinkability the settlement model's privacy guarantee depends on.

---

4. Node Discovery via Nostr

Node operators announce availability by publishing signed Nostr events to public relays. The ARFL client subscribes to these relays and maintains a live index of available nodes, updated continuously as new events arrive. No central directory exists. No registration is required.

4.1 Node Announcement Event

ARFL defines a custom Nostr event kind 30078 in the parameterised replaceable event range (30000–39999 per NIP-01), allowing relays to replace stale announcements by the same pubkey and 'd' tag. Node software republishes this event every 60 seconds. The deposit and its Fedimint attestation are included so that compliant clients can verify collateral cryptographically rather than trusting a self-reported number:

```
{
"kind": 30078,
"pubkey": "<node_nostr_pubkey_hex>",
"created_at": <unix_timestamp>,
"tags": [
["d", "<unique_node_id>"],
["wg_pubkey", "<wireguard_pubkey_base64>"],
["endpoint", "<ip_address>:<port>"],
["lnurl", "<lnurl_pay_address>"],
["deposit_sats", "<fedimint_attested_deposit>"],
["deposit_sig", "<fedimint_3of5_schnorr_sig_over_pubkey|deposit|ts>"],
["deposit_confirmed_at", "<unix_timestamp>"],
["upload", "<mbps>"],
["download", "<mbps>"],
["load", "<current_user_count>"],
["capacity", "<max_user_count, must be <= floor(deposit_sats/10000)>"],
["role", "both|entry|exit"],
["version", "<arfl_node_version>"]
],
"content": "",
"id": "<sha256_of_canonical_serialisation>",
"sig": "<schnorr_signature>"
}
```

4.2 Hub Announcement Event

Hubs publish their own Nostr announcement using event kind 30079. Because hub pricing is no longer protocol-fixed (Section 6), price_sats is simply whatever the hub has chosen to charge — there is no protocol value to validate it against. The client reads all available hubs and selects automatically based on reputation and uptime, or allows manual selection by the user:

```
{
"kind": 30079,
"pubkey": "<hub_nostr_pubkey_hex>",
"tags": [
["d", "<unique_hub_id>"],
["name", "<hub_name>"],
["lnurl", "<hub_lightning_address>"],
["price_sats", "<hub_chosen_bundle_price, no protocol floor or ceiling>"],
["uptime", "<percentage>"],
["version", "<arfl_hub_version>"]
],
"content": "",
"id": "<sha256_of_canonical_serialisation>",
"sig": "<schnorr_signature>"
}
```

4.3 Node Selection Algorithm

The client selects an entry and exit node pair on each new session. Deposit attestation is now verified as an explicit precondition, before scoring — a node whose Fedimint attestation is missing, invalid, below the 10,000-sat minimum, or whose claimed capacity exceeds floor(deposit_sats / 10,000) is never presented as selectable:

```
0. Verify Fedimint deposit_sig for every candidate node.
Discard: unverifiable sig, deposit_sats < 10,000,
or capacity > floor(deposit_sats / 10,000).
1. Filter: status=online, load < capacity, version compatible.
2. Score each node:
score = (capacity - load) / capacity
* upload_mbps / 50
* uptime_ratio
3. Select entry node: highest score, role=entry|both.
4. Select exit node: highest score, role=exit|both,
different operator pubkey to entry.
5. Pairing established client-side only.
Neither node is informed of the other.
```

4.4 Collusion Prevention

The two-node design prevents collusion by construction. Entry and exit nodes are selected independently and identified only by pseudonymous Nostr keypairs. They never communicate directly. All routing decisions are made unilaterally by the client. An operator running multiple nodes under different keypairs cannot coordinate a fraudulent session because the nodes have no awareness of each other’s involvement. Under the leading e-cash settlement design (Section 6), this is reinforced further: colluding entry and exit nodes cannot even correlate a session via payment timing, since Fedimint’s blind signatures make a mint operation unlinkable to its later redemption.

5. VPN Transport via WireGuard

WireGuard provides the encrypted transport layer. ARFL node software manages WireGuard programmatically using the wgctrl library (golang.zx2c4.com/wireguard/wgctrl, MIT licensed), which communicates with the WireGuard kernel module via generic netlink. The CLI is never used in production.

5.1 Session Establishment

A complete ARFL session proceeds as follows. The session token in step 2 is described in full in Section 10.3. Peer provisioning (steps 6–7) is also the point at which e-cash routing shares are handed to nodes as part of the settlement flow described in Section 6.

```
1. Client reads Nostr, selects entry + exit node pair (Sec. 4.3).
2. Client generates session token:
token = BLAKE2s(wg_pubkey || timestamp || nonce)
3. Client presents to hub: (wg_pubkey, session_token)
4. Hub verifies signed GB receipt: balance > 0, not expired.
5. Hub returns: (entry_endpoint, exit_endpoint, session_id)
6. Hub adds client peer on entry node via wgctrl.
7. Hub adds entry node as allowed peer on exit node.
8. Client initiates WireGuard handshake to entry node.
9. Noise_IKpsk2 handshake completes -- tunnel established.
10. Traffic flows: client -> entry -> exit -> internet.
11. DNS set to 9.9.9.9 (Quad9) within tunnel -- no DNS leak.
```

5.2 Per-Peer Byte Accounting

WireGuard maintains monotonic 64-bit byte counters per peer, updated per-packet in the kernel data path. The ARFL node daemon polls these counters every five seconds via wgctrl. Under the fallback settlement model (Section 6) these counters are the basis of daily settlement; under the leading e-cash model they remain the basis of GB-served accounting and refund sizing, but no longer gate node payment itself, since payment happens synchronously at purchase.

5.3 Quota Enforcement

Quota is enforced in 256 MB kernel slabs via nftables. As each slab is consumed the node refreshes the quota element. When the GB balance reaches zero the peer is removed. This two-layer approach eliminates the polling overshoot window: the kernel enforces the hard stop independently of the polling cycle.

5.4 Key Management and Device Portability

The user’s WireGuard private key is generated locally on first launch and stored in the device secure enclave. It is never transmitted to any ARFL component. The public key is the user’s sole network identity, linked to their GB receipt. To migrate to a new device, the user exports their private key as a QR code from the original device and imports it on the new device. If the key is lost, the balance is irrecoverable by design — the same property that prevents account recovery prevents identity linkage.

6. Payment Architecture

ARFL introduces a pay-per-use VPN payment model in which the hub sets its own bundle price freely — there is no protocol floor, no ceiling, and no protocol fee, in v1 or in any future version. The entire price a user pays is split evenly among the node operators who route that session. This section describes the purchase flow, the leading peer-to-peer settlement design, and the hub-mediated fallback that is retained in full as a documented contingency.

The hub sets its own price; the protocol enforces only how that price is divided among the nodes that route the session. There is no protocol floor, no ceiling, and no protocol fee — a fee would require a payment destination the protocol could verify and enforce, which would reintroduce the exact centralisation this protocol exists to eliminate.

6.1 Protocol Fixed Values

Parameter	Value	Rationale
Bundle size	250 GB	Standard unit of bandwidth
Bundle price	Set freely by each hub	No protocol floor or ceiling
Routing split (v1)	50 / 50 (entry / exit)	Protocol collects no fee, ever
Protocol fee	0%, always	No payment destination the protocol could ever verify or enforce
Node deposit	10,000 sats, fixed	Fedimint-attested; backs user refunds
Node capacity cap	floor(deposit_sats / 10,000)	Bounds concurrent-refund liability at the source
Hub deposit	Self-selected, publicly declared	Fallback-model only; hub bears its own under-deposit risk
Bundle expiry	30 days	Encourages active use
Min bandwidth	50 Mbps up/down	Baseline service quality
Ping interval	60 seconds	Node health monitoring
Offline trigger	30 minutes	Refund activation threshold
DNS resolver	9.9.9.9 (Quad9)	Privacy-preserving resolver
Settlement (primary)	Fedimint e-cash, minted at purchase	No hub custody window; nodes redeem on own schedule
Settlement (fallback)	Daily, per GB served	Used when e-cash settlement is unavailable to a hub

Table 1. ARFL protocol fixed values.

6.2 Purchase Flow

The complete purchase flow from user payment to GB credit and node settlement. All amounts are denominated in satoshis at the live market rate at the moment of purchase:

```
1. User selects hub from Nostr discovery; sees hub-set price_sats.
2. App prompts for Lightning refund address (optional LNURL).
Users without one waive automated refund rights.
3. App sends to hub: (wg_pubkey, refund_lnurl)
4. Hub generates BOLT11 invoice at its own set price:
amount = hub_price_sats at live rate
description = 'ARFL 250GB bundle'
expiry = 3600 seconds
5. User pays invoice from any Lightning wallet.
6. Hub receives payment preimage -- confirms settlement.
7. Hub issues signed GB receipt to client (250 GB, expiry,
price_paid_sats, hub_sig) -- the bundle contract.
8. Hub mints Fedimint e-cash for price_paid_sats, blind-signed,
and hands the notes to the client synchronously.
9. Client verifies it received the correct total note value.
10. Receipt + e-cash notes stored on client device. Not on hub.
11. App displays: '250 GB available. Expires <date>.'
```

Active sessions are unaffected by hub downtime. WireGuard tunnels remain open independent of hub availability. Only new purchases pause until the hub returns.

6.3 Node Settlement: Peer-to-Peer via Fedimint E-Cash

Rather than the hub batching node payments once a day, the hub mints blind-signed Fedimint e-cash for the exact bundle price at purchase time and hands it to the client immediately. The client splits the notes per the fixed ratio and pays entry and exit directly at session setup. Nodes redeem the notes for real sats whenever they choose — their own liquidity timing, not a protocol concern.

- **No hub custody window.** A daily-batch model would have the hub holding all node payment funds for up to 24 hours before settling, requiring signed price attestation, byte-counter cross-checks, and a hub deposit-and-slash mechanism just to keep the hub honest. None of that is needed if the hub never holds the funds in the first place.
- **Closes a correlation risk that a simpler design would leave open.** A real-time, per-session Lightning payment would give entry and exit nodes a precise payment timestamp and amount per session — if those two nodes ever collude or are subpoenaed together, they could correlate a session across both hops. Fedimint's blind signatures mean the federation cannot link a mint operation to its later redemption, closing both the custody gap and the correlation risk at once.
- **Depends on guardian independence.** This privacy property only holds if Fedimint guardians are non-partisan and independent of any hub operator (Section 3.4) — otherwise a hub could in principle recognise its own notes at redemption.

As an operational safeguard, the protocol also specifies a hub-mediated fallback (Section 6.3a), used when e-cash settlement is not available to a given hub: hub-signed price attestation, each node's own byte counters, and a Fedimint slash against the hub's self-declared deposit.

6.3a Fallback: Hub-Mediated Settlement

When e-cash settlement is not available to a hub, the hub accumulates node operator earnings and settles daily via Lightning to the LNURL published in each node's Nostr announcement, against a signed price attestation it cannot retract:

```
1. At peer provisioning, hub attaches a signed price attestation:
{ session_id, price_paid_sats, hub_sig }
Node refuses to serve if missing or invalid.
2. Hub tracks GB served per node via WireGuard byte counters
(polled every 5s).
3. Hub applies the even split against its own attested price,
pays node LNURLs daily.
4. Node independently computes expected payout (own byte
counters x attested price x split ratio) and compares to
actual payment received.
5. Shortfall -> node submits proof to Fedimint -> claim against
the HUB's own declared deposit (not the node deposit).
```

The hub deposit that backs this fallback claim is self-selected and publicly declared — the protocol cannot verify a hub's true revenue or volume, so responsibility for adequate collateral sits with the hub, the only party able to judge its own risk. This deposit is unrelated to, and does not replace, the protocol-fixed node operator deposit described in Section 7.

6.4 Wallet Sovereignty

ARFL places no requirements on the type of Lightning wallet used by users or node operators. Custodial wallets, non-custodial wallets, KYC and KYC-free wallets are all compatible — any wallet capable of paying a BOLT11 invoice and receiving to an LNURL is sufficient. Because some popular wallets (e.g. BlueWallet) lack a native self-custodial Lightning implementation, ARFL accepts a user-supplied LNURL from any wallet the user chooses rather than mandating a specific one; a cross-platform node daemon exposing a localhost REST API is in development to widen wallet compatibility further. Privacy and custody decisions remain the sole responsibility of the wallet holder. ARFL is a protocol, not a custodian.

7. Deposit and Refund Model

ARFL now distinguishes two structurally different deposit types, both Fedimint-custodied but serving different purposes. They are not interchangeable and should not be conflated.

Deposit type	Sizing	Purpose
Node operator deposit	Protocol-fixed minimum: 10,000 sats	Backs user refund claims; Fedimint-attested via Schnorr signature
Hub deposit	Self-selected, publicly declared	Fallback-model only — backs node split-shortfall claims if e-cash settlement is not used

Table 2. Two distinct Fedimint-custodied deposit types.

Each deposit is sized by the party able to judge the relevant risk: the protocol can verify a node deposit via Fedimint attestation and so sets a real minimum; it can never verify a hub's true revenue or volume, so hub deposit sizing is left to the hub, which bears the risk of under-depositing.

7.1 Fedimint Federation and Deposit Custody

The ARFL protocol team operates a Fedimint federation of five nodes in separate global jurisdictions. Three-of-five threshold signatures are required to release or slash any deposit. From the node operator's perspective the deposit address is indistinguishable from any Lightning address. All deposits are collateral, not a fee: the design is intentionally non-refundable on abandonment (Section 7.4), and there is no operator-withdrawal pathway — a node's signing-key compromise is a reputation and griefing risk to that operator, not a fund-theft risk to the protocol or its users.

In future protocol versions, federation membership will be opened to community operators, progressively decentralising deposit custody consistent with the broader ARFL architecture. Guardians must be non-partisan and operationally independent of any hub operator (Section 3.4) — a hard requirement for the e-cash settlement model's privacy guarantee to hold, not merely good practice.

7.2 Node Capacity Cap

A node's advertised maximum concurrent-user capacity can never exceed floor(deposit_sats / 10,000), enforced client-side against the node's real, Fedimint-attested deposit — not its self-reported claim. A node depositing 10,000 sats may advertise capacity 1; a node wanting to serve roughly 100 concurrent users needs 1,000,000 sats deposited. This bounds the risk directly: a node could otherwise advertise unlimited capacity behind a flat, tiny deposit, and if a busy node serving many concurrent users went offline, the resulting refund liability could exceed what its deposit could actually cover. Over-claiming capacity now simply makes a node unselectable by compliant clients, rather than a problem discovered only after a failure.

7.3 Automated Refund Flow

The hub (or, under the e-cash model, the client directly) monitors every active node via signed ping events every 60 seconds. When a node is offline for 30 consecutive minutes, the following automated sequence executes without human intervention. Reliability is binary: either the service was provided or it was not. No partial refunds are issued.

```
1. 30 consecutive signed failed ping timestamps recorded.
2. Refund for the undelivered portion is paid to the user's
refund_lnurl directly from that node's own Fedimint deposit
-- no hub float involved under the e-cash model.
3. Node deposit balance reduced accordingly.
4. If deposit hits zero: node auto-delisted from Nostr routing.
```

Under the hub-mediated fallback model only, the hub fronts the refund from its own working capital first, then reclaims from the offline node's deposit via Fedimint proof-of-offline plus a Lightning refund proof — the hub's own liquidity management for this float is entirely its own business decision, out of protocol scope. Refunds are denominated and paid in satoshis at the time of refund; the protocol makes no guarantee of USD equivalence, consistent with paying in Bitcoin in the first place.

7.4 Deposit Top-Up and Protocol Treasury

When a node deposit falls below the minimum, the node software automatically alerts the operator and generates a Lightning invoice for top-up. The federation credits the deposit and the node is immediately relisted on Nostr — the only manual action in the entire refund lifecycle.

Node operators who abandon the network forfeit their remaining deposit. After 90 days of inactivity the federation automatically transfers the remaining balance to the ARFL protocol treasury, which funds protocol maintenance and development. The deposit is a participation commitment, not a reversible payment. At the locked 10,000-sat minimum, and given that operators typically recoup this within days of joining, forfeiture on abandonment is an acceptable and intentional tradeoff — and, per Section 11, one of the only two ways any sats ever reach an ARFL-controlled address at all.

---

8. Webhook Architecture

The ARFL protocol publishes real-time events to subscribed hubs via encrypted Nostr direct messages (NIP-04). Hubs subscribe by publishing a kind 30080 Nostr event specifying their webhook endpoint and pubkey. This enables hubs to monitor network state, automate responses, and build differentiated user experiences on top of a shared protocol.

8.1 Event Types

Event	Trigger	Hub Action
NODE_OFFLINE	Node fails ping	Start 30-min timer
NODE_ONLINE	Node resumes ping	Cancel timer if active
REFUND_TRIGGERED	30-min offline confirmed	Pay user (or verify node-funded refund)
DEPOSIT_LOW	Deposit below threshold	Alert node operator
DEPOSIT_DEPLETED	Deposit reaches zero	Delist node from routing
GB_BALANCE_LOW	Receipt below 25 GB	Push purchase prompt
ECASH_MINTED	Notes minted at purchase	Log for audit / analytics
SESSION_STARTED	WireGuard tunnel established	Log session start

Event	Trigger	Hub Action
SESSION_ENDED	Tunnel closed	Update GB accounting
SETTLEMENT_SENT	Node payment sent (either model)	Reconcile ledger

Table 3. ARFL webhook event types and recommended hub actions.

8.2 Hub Subscription Event

```
{
"kind": 30080,
"pubkey": "<hub_nostr_pubkey>",
"tags": [
["d", "<hub_id>"],
["webhook", "https://hub.example.com/arfl/events"],
["events", "NODE_OFFLINE,REFUND_TRIGGERED,
GB_BALANCE_LOW,DEPOSIT_LOW"]
],
"content": "",
"sig": "<schnorr_signature>"
}
```

Protocol events are delivered encrypted to the hub’s Nostr pubkey via NIP-04 direct messages. Hubs may additionally specify an HTTPS webhook endpoint for direct POST delivery. Both delivery mechanisms are supported for redundancy.

9. Privacy Architecture

ARFL is designed so that no single entity accumulates sufficient information to reconstruct a user session:

Component	Knows	Does Not Know
Entry node	User source IP	Destination, user identity
Exit node	Traffic destination	User IP, user identity
Hub	WireGuard pubkey, receipt sig, price_paid_sats	Browsing, node pairing, IP
Nostr relay	Node and hub announcements	User activity, sessions
Fedimint	Deposit balances; mint amounts (not redemption identity)	Sessions, user identity, mint-to-redeem link
Anyone	Nothing complete	Full session picture

Table 4. Information compartmentalisation across ARFL components (v1).

DNS queries are routed through the encrypted WireGuard tunnel to Quad9 (9.9.9.9) — a privacy-preserving resolver with a published no-logs policy. This is enforced in the WireGuard interface configuration delivered at session start. The exit node never sees DNS queries. No DNS leak is architecturally possible.

The hub stores only the WireGuard public key, receipt signature, expiry timestamp, price_paid_sats, and refund LNURL — none of which constitute identity. Under this settlement model the hub additionally never holds node-payment funds at all, narrowing its role further. Because the hub is open source, this claim is verifiable by inspection, not merely asserted in a privacy policy.

10. Security Model

10.1 Threat Model

ARFL protects against:

- ISP traffic surveillance and deep packet inspection.
- Network-level IP address tracking and logging.
- DNS query surveillance and domain-level logging.
- Identity-linked browsing history.
- Legal compulsion of a central VPN provider.
- Server seizure by authorities.
- Corporate acquisition or policy change.
- Payment-timing correlation between colluding entry and exit nodes.

ARFL does not protect against:

- Malware or compromise of the user's own device.
- Browser fingerprinting techniques.
- Services the user voluntarily authenticates into.
- A coordinated adversary controlling a majority of the node network simultaneously.

10.2 Sybil Resistance

An attacker attempting to control a large fraction of the node network must fund a 10,000-sat deposit per node identity — Fedimint-attested, not self-reported — all held at risk of depletion through refund obligations. This figure was calibrated once against an original $10 Sybil-resistance reference point at a roughly $63k/BTC price at time of decision, and is now fixed in sats going forward: no live price feed, no ongoing USD peg, consistent with the protocol's broader sats-standardisation principle. At 1,000 nodes this requires 10,000,000 sats in capital, all independently verifiable via Fedimint attestation rather than trusted on a node's own claim. The node capacity cap (Section 7.2) additionally bounds the refund exposure any single Sybil identity can create, closing a gap the deposit minimum alone did not solve.

10.3 Replay Attack Prevention

Each session request includes a token derived from the user's WireGuard public key, the current timestamp, and a random nonce: token = BLAKE2s(wg_pubkey || timestamp || nonce). The hub maintains a short-lived token cache and rejects duplicate tokens within a five-minute window, preventing replay of valid session requests.

10.4 WireGuard Traffic Analysis Resistance

WireGuard is silent by design: it does not respond to unauthenticated packets and sends no response to packets that fail authentication. This makes ARFL nodes invisible to port scanners and resistant to active probing. The fixed-suite cryptography eliminates cipher negotiation as an attack surface.

---

11. Trust Model

ARFL is designed to require no trust in any single party. Each security property is enforced by cryptographic guarantees, economic incentives, or open source verifiability:

Property	Enforcement
Encrypted transport	WireGuard ChaCha20-Poly1305 — cryptographic
No user identity	WireGuard keypair — locally generated, never transmitted
No session logs	Architecture — no component sees full session
Node availability	Security deposit — economic penalty for downtime
Payment proof	BOLT11 preimage — cryptographic proof of payment
Node authenticity & deposit size	Nostr Schnorr signature + Fedimint attestation — cryptographic
Quota enforcement	nftables kernel quota — OS enforced

Property	Enforcement
Deposit custody	Fedimint threshold signatures — 3 of 5 required
Session-correlation resistance	Fedimint blind signatures — unlinkable mint-to-redeem
No protocol fee	No ARFL-controlled payment destination exists, by design — structural
No DNS leak	WireGuard DNS config — architectural
Hub compliance	Client validation + open source + community audit

Table 5. Trust properties and enforcement mechanisms.

The hub requires bounded trust: under this settlement model it never holds node-payment funds at all, and even under the fallback model it cannot access browsing history, cannot deanonymise users, and cannot steal funds beyond a provably-detectable shortfall claimable against its own declared deposit. Its source code is open and auditable. The Fedimint federation requires trust in the protocol team in v1, with progressive decentralisation to community operators in subsequent versions, and a hard governance requirement that guardians remain independent of any hub operator.

12. Node Operator Economics

ARFL node operators earn Bitcoin passively by contributing bandwidth to the network. The economic model is analogous to Bitcoin proof-of-work: operators contribute a resource, are rewarded proportionally to contribution, and earn nothing when offline. The Airbnb parallel is also instructive: node operators list their bandwidth as hosts list their properties — they do not receive free VPN access any more than an Airbnb host receives free stays elsewhere. Users and node operators are entirely separate participant classes.

Because hubs now set their own bundle price freely, node earnings scale directly with whatever a given hub charges rather than being pinned to a protocol-fixed floor. The figures below are therefore illustrative, built around a representative hub price roughly equivalent to $5 per 250 GB bundle — an individual hub may charge more or less, and node earnings move with it, split 50/50 between entry and exit under the locked v1 model.

Node Activity	Simultaneous Users	GB Served / Month	Est. Monthly Earnings (illustrative)
Quiet	~10	~2 TB	~$40 in sats (50% of ~$80 revenue)
Active	~50	~10 TB	~$200 in sats
Busy	~100	~20 TB	~$400 in sats

Table 6. Illustrative node operator monthly earnings by activity level, at an example $5/250GB hub price. Actual earnings scale with each hub's own chosen price.

The initial cost to participate as both node operator and user is modest: a 10,000-sat deposit (roughly the cost of a coffee at typical exchange rates, illustrative only) to join the network, plus whatever a chosen hub charges for a bundle to use the VPN. At the “Busy” tier above, a node recoups its full deposit in well under a day.

Two-hop routing does not double per-node bandwidth cost, even though it doubles aggregate network bandwidth for a given bundle. Each individual node — entry or exit — only ever relays that bundle's traffic exactly once, identical to what a single-hop node would bear; the 2x figure is a network-wide statistic, not any one operator's expense. This is why an even split across routing roles is fair compensation for being one privacy hop, not a subsidy for an inflated cost that does not actually exist at the node level.

Nodes with faster connections serve more simultaneous users and earn proportionally more at the same per-GB share of whatever the hub charges. The market self-selects for quality without tiered pricing or central intervention. The minimum requirements to run a node are deliberately low:

- 50 Mbps upload and download minimum.
- Any Lightning wallet supporting LNURL — no full Lightning node required.
- 10,000-sat security deposit, Fedimint-attested.
- 80% uptime commitment.
- ARFL node software (open source, Go).

13. Hub Economics

Hub revenue is entirely out-of-protocol scope. The protocol's only guarantee is that whatever price a hub attests for a session passes through in full to the routing node operators — there is no protocol-defined hub cut, no mandated margin, and no mechanism by which the protocol could size, verify, or enforce hub monetisation even if one were wanted. This mirrors the AWS or Stripe relationship: the infrastructure layer has no say in how a business built on top of it prices its own product.

Because the full bundle price passes through to node operators (Section 6), nothing in the protocol layer is reserved for the hub. The protocol can no more verify a hub's true revenue than it can verify its transaction volume, so hub monetisation is left entirely external, the same way hub deposit sizing is left to the hub. A hub is free to monetise however it chooses — a subscription layer, advertising, a separate service fee charged outside the BOLT11 bundle flow, bundling with other products, or accepting thinner margins funded elsewhere — entirely outside what the protocol governs or observes.

Hubs compete on brand, reliability, and service quality rather than on the underlying node network, since all compliant hubs share the same node pool and therefore the same routing infrastructure quality. Differentiation comes from the user experience built on top of the protocol: how quickly a hub responds to node failures, how well it notifies users, what premium features it offers, and the trust its brand commands in the community. There is no protocol-enforced floor or ceiling left to police — user protection now relies on ordinary market competition, since users can see and compare price_sats across hubs at the point of discovery (Section 4.2).

---

14. Why No Token

Every prior decentralised privacy network introduced a native token: Mysterium (MYST), Sentinel (DVPN), Orchid (OXT). More recently, El Tor has explored Lightning-incentivized Tor relaying with time-based payments, but still relies on a centralised Directory Authority for relay discovery. Tokens introduce speculation over utility, regulatory uncertainty across jurisdictions, participation barriers for users unfamiliar with token acquisition, and protocol complexity that distracts from the underlying mission. A centralised discovery authority, whatever its payment rail, reintroduces exactly the single point of failure and pressure this protocol exists to remove.

ARFL uses Bitcoin — the monetary network that already exists, already works, and is already held by the community most aligned with ARFL's values. The Lightning Network provides the micropayment infrastructure that makes per-bundle pricing practical at negligible transaction cost, and Fedimint extends this to trustless, unlinkable settlement between users and node operators. No new token is introduced. No ICO is conducted. No token treasury exists. The incentive mechanism is Bitcoin, and Bitcoin alone — and, per Section 6 and Section 11, the protocol itself never takes a cut of it, either.

15. Conclusion

We have proposed a decentralised privacy protocol that requires no trust in any central authority. By aggregating WireGuard for encrypted transport, Nostr for pseudonymous discovery, the Bitcoin Lightning Network for payment, and Fedimint for trustless deposit custody and peer-to-peer settlement, ARFL creates a privacy network where node operators are rewarded in proportion to whatever a hub actually charges, users pay only for what they use, hubs compete freely on price and service rather than infrastructure, and no single point of failure can compromise the network.

The protocol collects no fee, in any version, under any settlement model — this is not an oversight but an architectural constraint. Tracing through every mechanical way a protocol-level fee could be collected (a direct payment, routing through Fedimint, a dedicated treasury) shows all three require a concrete payment destination, revenue verification, and an enforcement authority: the exact centralisation this protocol's own thesis exists to eliminate. Any future funding for ARFL's stewards has to be solved at the business layer — running an ordinary compliant hub, adjacent tooling, grants — never by giving the protocol layer a revenue mechanism of its own.

The architecture makes two clean separations. Node operators interact directly with the protocol — contributing bandwidth, earning Bitcoin in full proportion to a hub's chosen price, with no hub relationship required beyond routing. Hubs build products on top of the protocol — setting their own price, providing user experiences, competing on brand, monetising however they choose, with no infrastructure to own and no protocol cut taken from underneath them. The protocol itself owns nothing, controls nothing, and requires no permission to build on.

The user's WireGuard keypair — generated locally, stored in the device secure enclave, never transmitted — is their sole identity. The network is robust in its simplicity. Node operators require only bandwidth and a Lightning wallet. Users require only satoshis and the ARFL application. The protocol requires only open source code, cryptographic guarantees, and economic incentive.

Privacy is not a product. It is a right. ARFL is the infrastructure for that right.

References

[1] D.J. Bernstein. "Curve25519: New Diffie-Hellman Speed Records." PKC 2006. https://cr.yp.to/ecdh/curve25519-20060209.pdf

[2] Y. Nir, A. Langley. "ChaCha20 and Poly1305 for IETF Protocols." RFC 8439. 2018. https://datatracker.ietf.org/doc/html/rfc8439

[3] Z. Donenfeld. "WireGuard: Next Generation Kernel Network Tunnel." NDSS 2017. https://www.wireguard.com/papers/wireguard.pdf

[4] WireGuard control library. wgctrl-go. MIT License. https://github.com/WireGuard/wgctrl-go

[5] fiatjaf et al. "Nostr Protocol — NIP-01: Basic protocol flow." https://github.com/nostr-protocol/nostr/blob/master/01.md

[6] fiatjaf et al. "Nostr Protocol — NIP-04: Encrypted Direct Messages." https://github.com/nostr-protocol/nostr/blob/master/04.md

[7] fiatjaf et al. "Nostr Protocol — NIP-19: bech32-encoded entities." https://github.com/nostr-protocol/nostr/blob/master/19.md

[8] J. Poon, T. Dryja. "The Bitcoin Lightning Network." https://lightning.network/lightning-network-paper.pdf, 2016.

[9] Lightning Network Specification. BOLT11: Invoice Protocol. https://github.com/lightning/bolts/blob/master/11-payment-encoding.md

[10] LNURL Specification. https://github.com/lnurl/luds

[11] Fedimint. Federated Chaumian Mints. https://github.com/fedimint/fedimint

[12] D. Chaum. “Blind Signatures for Untraceable Payments.” CRYPTO ’82, 1983.

[13] T. Perrin. “The Noise Protocol Framework.” https://noiseprotocol.org/noise.html, 2018.

[14] S. Nakamoto. “Bitcoin: A Peer-to-Peer Electronic Cash System.” https://bitcoin.org/bitcoin.pdf, 2008.

[15] Quad9. Privacy-preserving DNS resolver. https://www.quad9.net

[16] Mysterium Network. Decentralised VPN Protocol. https://github.com/mysteriumnetwork/node

[17] Sentinel. dVPN Protocol. https://github.com/sentinel-official

[18] BIP-340. Schnorr Signatures for secp256k1. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki